TOPIC 4 : DYNAMIC PROGRAMMING

1. Dice Throw Problem

main.py — Share — Run — Output

```python
def dice_throw(num_sides, num_dice, target):
    # dp[d][s] = number of ways to get sum s using d dice
    dp = [[0] * (target + 1) for _ in range(num_dice + 1)]

    # 0 dice can make sum 0 in exactly 1 way
    dp[0][0] = 1

    # Fill the DP table
    for d in range(1, num_dice + 1):
        for s in range(1, target + 1):
            for face in range(1, num_sides + 1):
                if s - face >= 0:
                    dp[d][s] += dp[d - 1][s - face]

    return dp[num_dice][target]


# Test Case 1
num_sides = 6
num_dice = 2
target = 7
print("Test Case 1:")
print("Number of ways to reach sum 7:", dice_throw(num_sides
    , num_dice, target))

# Test Case 2
```

```
Test Case 1:
Number of ways to reach sum 7: 6
Test Case 2:
Number of ways to reach sum 10: 6

=== Code Execution Successful ===
```

2. Assembly Line Scheduling

main.py — Share — Run — Output

```python
def assembly(a1,a2,t1,t2,e1,e2,x1,x2):
    n=len(a1)
    f1=e1+a1[0]
    f2=e2+a2[0]

    for i in range(1,n):
        f1,f2=min(f1+a1[i],f2+t2[i-1]+a1[i]),min(f2+a2[i],f1
            +t1[i-1]+a2[i])

    return min(f1+x1,f2+x2)

a1=[4,5,3,2]
a2=[2,10,1,4]
t1=[7,4,5]
t2=[9,2,8]

print("Minimum time:",assembly(a1,a2,t1,t2,10,12,18,7))
```

```
Minimum time: 35

=== Code Execution Successful ===
```

3.Three-Line Scheduling

main.py | Share | Run | Output

```
a = [
    [5, 9, 3],
    [6, 8, 4],
    [7, 6, 5]
]

t = [
    [0, 2, 3],
    [2, 0, 4],
    [3, 4, 0]
]

n = 3

# DP table
dp = [[0] * n for _ in range(3)]

# First station
for i in range(3):
    dp[i][0] = a[i][0]

# Remaining stations
for j in range(1, n):
    for i in range(3):
        dp[i][j] = min(dp[k][j-1] + t[k][i]
                       for k in range(3)) + a[i][j]
```

```
Minimum production time: 17

=== Code Execution Successful ===
```

4.Travelling Salesperson Problem (TSP)

main.py | Share | Run | Output

```
def tsp(city, count, total):
    global ans
    if count == n:
        ans = min(ans, total + a[city][0])
        return

    for i in range(n):
        if not visited[i]:
            visited[i] = 1
            tsp(i, count + 1, total + a[city][i])
            visited[i] = 0

n = 4
a = [
    [0,10,15,20],
    [10,0,35,25],
    [15,35,0,30],
    [20,25,30,0]
]

visited = [0] * n
visited[0] = 1
ans = 9999

tsp(0, 1, 0)

```

```
Minimum path distance: 80

=== Code Execution Successful ===
```

5.TSP with Five Cities

main.py

```python
from itertools import permutations

cities = ['A', 'B', 'C', 'D', 'E']

d = [
    [0,10,15,20,25],
    [10,0,35,25,30],
    [15,35,0,30,20],
    [20,25,30,0,15],
    [25,30,20,15,0]
]

best = 9999
route = None

for p in permutations(range(1,5)):
    path = (0,) + p + (0,)
    cost = sum(d[path[i]][path[i+1]] for i in range(5))

    if cost < best:
        best = cost
        route = path

print("Shortest route:", " -> ".join(cities[i] for i in route))
print("Total distance:", best)
```

Output

```
Shortest route: A -> B -> D -> E -> C -> A
Total distance: 85

=== Code Execution Successful ===
```

6.Longest Palindromic Substring

main.py

```python
def longest_palindrome(s):
    start = end = 0

    def expand(l, r):
        while l >= 0 and r < len(s) and s[l] == s[r]:
            l -= 1
            r += 1
        return l + 1, r - 1

    for i in range(len(s)):
        l1, r1 = expand(i, i)
        l2, r2 = expand(i, i + 1)

        if r1 - l1 > end - start:
            start, end = l1, r1
        if r2 - l2 > end - start:
            start, end = l2, r2

    return s[start:end + 1]

s = input("Enter string: ")
print("Longest palindromic substring:", longest_palindrome(s))
```

Output

```
Enter string: babad
Longest palindromic substring: bab

=== Code Execution Successful ===
```

7.Longest Substring Without Repeating Characters

main.py | Share | Run | Output

```python
def longest_substring(s):
    seen = set()
    left = 0
    ans = 0

    for right in range(len(s)):
        while s[right] in seen:
            seen.remove(s[left])
            left += 1

        seen.add(s[right])
        ans = max(ans, right - left + 1)

    return ans

s = input("Enter string: ")
print("Length of longest substring:", longest_substring(s))
```

```
Enter string: abcabcbb
Length of longest substring: 3

=== Code Execution Successful ===
```

8.Word Break Problem

main.py | Share | Run | Output

```python
def word_break(s, words):
    dp = [False] * (len(s) + 1)
    dp[0] = True

    for i in range(1, len(s) + 1):
        for word in words:
            if i >= len(word) and dp[i - len(word)]:
                if s[i-len(word):i] == word:
                    dp[i] = True
                    break

    return dp[len(s)]

s = input("Enter string: ")
words = input("Enter dictionary words: ").split()

print("Can be segmented:", word_break(s, words))
```

```
Enter string: leetcode

leet code
Enter dictionary words: Can be segmented: False

=== Code Execution Successful ===
```

9.Word Break Problem

main.py Share Run Output

```python
def word_break(s, words):
    dp = [False] * (len(s) + 1)
    dp[0] = True

    for i in range(1, len(s) + 1):
        for word in words:
            if i >= len(word) and dp[i - len(word)]:
                if s[i-len(word):i] == word:
                    dp[i] = True
                    break

    return dp[len(s)]


words = {"i", "like", "sam", "sung", "samsung",
         "mobile", "ice", "cream", "icecream", "man", "go",
             "mango"}

s = input("Enter string: ")

if word_break(s, words):
    print("Output: Yes")
else:
    print("Output: No")
```

```
Enter string: ilike
Output: Yes

=== Code Execution Successful ===
```

10.Text Justification

main.py Share Run Output

```python
def justify(words, width):
    result = []
    i = 0

    while i < len(words):
        line = [words[i]]
        length = len(words[i])
        i += 1

        while i < len(words) and length + 1 + len(words[i])
            <= width:
            line.append(words[i])
            length += 1 + len(words[i])
            i += 1

        spaces = width - sum(len(w) for w in line)

        if i == len(words) or len(line) == 1:
            result.append(" ".join(line) + " " * (width -
                length))
        else:
            gaps = len(line) - 1
            extra = spaces // gaps
            rem = spaces % gaps

            text = ""
```

```
"This    is    an"
"example  of text"
"justification.  "

=== Code Execution Successful ===
```

11.Prefix-Suffix Word Filter

main.py

```python
class WordFilter:
    def __init__(self, words):
        self.words = words

    def f(self, pref, suff):
        for i in range(len(self.words) - 1, -1, -1):
            word = self.words[i]
            if word.startswith(pref) and word.endswith(suff):
                return i
        return -1


words = ["apple"]
wf = WordFilter(words)

print(wf.f("a", "e"))
```

Output

```
0

=== Code Execution Successful ===
```

12.Floyd’s All-Pairs Shortest Path

main.py

```python
INF = 999

n = 4
dist = [
    [0, 3, 8, -4],
    [INF, 0, 4, 1],
    [2, INF, 0, INF],
    [INF, 6, -5, 0]
]

print("Before applying Floyd's Algorithm:")
for row in dist:
    print(row)

# Floyd's Algorithm
for k in range(n):
    for i in range(n):
        for j in range(n):
            dist[i][j] = min(dist[i][j],
                             dist[i][k] + dist[k][j])

print("\nAfter applying Floyd's Algorithm:")
for row in dist:
    print(row)
```

Output

```
Before applying Floyd's Algorithm:
[0, 3, 8, -4]
[999, 0, 4, 1]
[2, 999, 0, 999]
[999, 6, -5, 0]

After applying Floyd's Algorithm:
[-7, -4, -9, -11]
[-2, 0, -4, -6]
[-5, -2, -7, -9]
[-10, -7, -12, -14]

Shortest path from City 1 to City 3: -9

=== Code Execution Successful ===
```

13.Floyd's Algorithm with Link Failure

main.py

```python
INF = 999

routers = ["A", "B", "C", "D", "E", "F"]
n = 6

dist = [
    [0, 1, 5, INF, INF, INF],
    [1, 0, 2, 1, INF, INF],
    [5, 2, 0, INF, 3, INF],
    [INF, 1, INF, 0, 1, 6],
    [INF, INF, 3, 1, 0, 2],
    [INF, INF, INF, 6, 2, 0]
]

def floyd(d):
    for k in range(n):
        for i in range(n):
            for j in range(n):
                d[i][j] = min(d[i][j], d[i][k] + d[k][j])
    return d

# Before link failure
d1 = [row[:] for row in dist]
floyd(d1)

print("Before link failure:")
```

Output

```
Before link failure:
Router A to Router F = 5
After link failure:
Router A to Router F = 8

=== Code Execution Successful ===
```

14.Floyd's Algorithm with Distance Threshold

main.py

```python
INF = 999

def floyd(d):
    n = len(d)

    print("Before:")
    for row in d:
        print(row)

    for k in range(n):
        for i in range(n):
            for j in range(n):
                d[i][j] = min(d[i][j], d[i][k] + d[k][j])

    print("\nAfter:")
    for row in d:
        print(row)

    return d


# Test Case A
d1 = [
    [0, 2, 3, INF],
    [INF, 0, INF, INF],
    [7, INF, 0, 1],
```

Output

```
Before:
[0, 2, 3, 999]
[999, 0, 999, 999]
[7, 999, 0, 1]
[6, 999, 999, 0]

After:
[0, 2, 3, 4]
[999, 0, 999, 999]
[7, 9, 0, 1]
[6, 8, 9, 0]
C to A = 7
Before:
[0, 4, 999, 5, 999]
[999, 0, 1, 999, 6]
[2, 999, 0, 3, 999]
[999, 999, 1, 0, 2]
[1, 999, 999, 4, 0]

After:
[0, 4, 5, 5, 7]
[3, 0, 1, 4, 6]
[2, 6, 0, 3, 5]
[3, 7, 1, 0, 2]
[1, 5, 5, 4, 0]
E to C = 5
```

15.Optimal Binary Search Tree (OBST)

main.py — Share — Run — Output

```python
def obst(keys, freq):
    n = len(keys)
    cost = [[0] * (n + 1) for _ in range(n + 1)]
    root = [[0] * (n + 1) for _ in range(n + 1)]

    for i in range(n):
        cost[i][i + 1] = freq[i]
        root[i][i + 1] = i

    for length in range(2, n + 1):
        for i in range(n - length + 1):
            j = i + length
            total = sum(freq[i:j])
            cost[i][j] = float('inf')

            for r in range(i, j):
                c = cost[i][r] + cost[r + 1][j] + total
                if c < cost[i][j]:
                    cost[i][j] = c
                    root[i][j] = r

    print("Cost Matrix:")
    for row in cost:
        print(row)

    print("\nRoot Matrix:")
```

```
Cost Matrix:
[0, 0.1, 0.4, 1.1, 1.7]
[0, 0, 0.2, 0.8, 1.4]
[0, 0, 0, 0.4, 1.0]
[0, 0, 0, 0, 0.3]
[0, 0, 0, 0, 0]

Root Matrix:
[0, 0, 1, 2, 2]
[0, 0, 1, 2, 2]
[0, 0, 0, 2, 2]
[0, 0, 0, 0, 3]
[0, 0, 0, 0, 0]

Minimum Cost: 1.7

=== Code Execution Successful ===
```

16.OBST Construction and Cost

main.py — Share — Run — Output

```python
def obst(keys, freq):
    n = len(keys)
    cost = [[0]*(n+1) for _ in range(n+1)]
    root = [[0]*(n+1) for _ in range(n+1)]

    for i in range(n):
        cost[i][i+1] = freq[i]
        root[i][i+1] = i

    for length in range(2, n+1):
        for i in range(n-length+1):
            j = i + length
            total = sum(freq[i:j])
            cost[i][j] = 999999

            for r in range(i, j):
                x = cost[i][r] + cost[r+1][j] + total
                if x < cost[i][j]:
                    cost[i][j] = x
                    root[i][j] = r

    print("Cost Matrix:")
    for row in cost:
        print(row)

    print("\nRoot Matrix:")
```

```
Cost Matrix:
[0, 4, 8, 20, 26]
[0, 0, 2, 10, 16]
[0, 0, 0, 6, 12]
[0, 0, 0, 0, 3]
[0, 0, 0, 0, 0]

Root Matrix:
[0, 0, 0, 2, 2]
[0, 0, 1, 2, 2]
[0, 0, 0, 2, 2]
[0, 0, 0, 0, 3]
[0, 0, 0, 0, 0]

Minimum Cost: 26
Root: 16

=== Code Execution Successful ===
```

17.Cat and Mouse Game

main.py Share Run Output

```python
from collections import deque

def cat_mouse_game(graph):
    n = len(graph)
    # result[mouse][cat][turn]
    result = [[[0]*2 for _ in range(n)] for _ in range(n)]
    degree = [[[0]*2 for _ in range(n)] for _ in range(n)]
    q = deque()

    for m in range(n):
        for c in range(n):
            degree[m][c][0] = len(graph[m])
            degree[m][c][1] = len([x for x in graph[c] if x
                != 0])

    # Mouse reaches hole -> Mouse wins
    for c in range(1, n):
        result[0][c][0] = result[0][c][1] = 1
        q.append((0, c, 0))
        q.append((0, c, 1))

    # Cat catches mouse -> Cat wins
    for i in range(1, n):
        result[i][i][0] = result[i][i][1] = 2
        q.append((i, i, 0))
        q.append((i, i, 1))
```

```
Example 1: 0
Example 2: 1

=== Code Execution Successful ===
```

18.Maximum Probability Path

main.py Share Run Output

```python
import heapq

def max_probability(n, edges, prob, start, end):
    graph = [[] for _ in range(n)]

    for i, (a, b) in enumerate(edges):
        graph[a].append((b, prob[i]))
        graph[b].append((a, prob[i]))

    best = [0] * n
    best[start] = 1
    pq = [(-1, start)]

    while pq:
        p, u = heapq.heappop(pq)
        p = -p

        if u == end:
            return p

        for v, w in graph[u]:
            new_p = p * w

            if new_p > best[v]:
                best[v] = new_p
```

```
Maximum probability: 0.25
Maximum probability: 0.3

=== Code Execution Successful ===
```

19.Unique Paths in a Grid

main.py Share Run Output

```python
def unique_paths(m, n):
    dp = [[1] * n for _ in range(m)]

    for i in range(1, m):
        for j in range(1, n):
            dp[i][j] = dp[i-1][j] + dp[i][j-1]

    return dp[m-1][n-1]


# Example 1
print("Example 1:", unique_paths(3, 7))

# Example 2
print("Example 2:", unique_paths(3, 2))
```

```
Example 1: 28
Example 2: 3

=== Code Execution Successful ===
```

20.Count Good Pairs

main.py Share Run Output

```python
def good_pairs(nums):
    count = 0
    freq = {}

    for num in nums:
        if num in freq:
            count += freq[num]
        freq[num] = freq.get(num, 0) + 1

    return count


# Example 1
nums = [1, 2, 3, 1, 1, 3]
print("Example 1:", good_pairs(nums))

# Example 2
nums = [1, 1, 1, 1]
print("Example 2:", good_pairs(nums))
```

```
Example 1: 4
Example 2: 6

=== Code Execution Successful ===
```

21.City with Minimum Reachable Cities

main.py | Share | Run | Output

```python
def find_city(n, edges, threshold):
    INF = 9999
    d = [[INF] * n for _ in range(n)]

    for i in range(n):
        d[i][i] = 0

    for a, b, w in edges:
        d[a][b] = d[b][a] = w

    # Floyd's Algorithm
    for k in range(n):
        for i in range(n):
            for j in range(n):
                d[i][j] = min(d[i][j], d[i][k] + d[k][j])

    ans = -1
    minimum = n

    for i in range(n):
        count = sum(d[i][j] <= threshold for j in range(n)
            if i != j)

        if count <= minimum:
            minimum = count
            ans = i
```

```
Example 1: 3
Example 2: 0

=== Code Execution Successful ===
```

22.Network Delay Time

main.py | Share | Run | Output

```python
import heapq

def network_delay(times, n, k):
    graph = [[] for _ in range(n + 1)]

    for u, v, w in times:
        graph[u].append((v, w))

    dist = [9999] * (n + 1)
    dist[k] = 0
    pq = [(0, k)]

    while pq:
        d, u = heapq.heappop(pq)

        if d > dist[u]:
            continue

        for v, w in graph[u]:
            nd = d + w
            if nd < dist[v]:
                dist[v] = nd
                heapq.heappush(pq, (nd, v))

    ans = max(dist[1:])
```

```
2
1
-1

=== Code Execution Successful ===
```